

Evaluador de expresiones con números duales

Luciano David Duarte

Análisis de Lenguajes de Programación
Licenciatura en Ciencias de la Computación – UNR

Abstract

Este trabajo implementa un DSL/EDSL pequeño para representar expresiones matemáticas, evaluarlas en un punto y calcular derivadas mediante números duales. La propuesta inicial estaba centrada en mostrar el comportamiento de los números duales; durante el desarrollo se extendió hacia una calculadora de expresiones con parser, modo interactivo, lectura de archivos, pretty printer, optimización conservadora, manejo explícito de errores y una mónada propia de evaluación. El resultado no intenta ser un sistema de álgebra computacional completo, sino una herramienta acotada para estudiar cómo se modela un lenguaje en Haskell usando tipos algebraicos, parser combinators, type classes y mónadas.

1 Introducción

El objetivo del proyecto es construir un lenguaje pequeño para escribir expresiones matemáticas y evaluarlas de forma segura. En lugar de trabajar directamente con strings durante toda la ejecución, el programa transforma la entrada en un árbol de sintaxis abstracta. A partir de ese árbol se pueden definir distintas interpretaciones: evaluación escalar, evaluación con números duales, impresión legible y optimización.

La diferenciación automática con números duales es el eje conceptual del trabajo. Si una expresión se evalúa reemplazando la variable por un número dual de la forma $x + \varepsilon$, el resultado contiene simultáneamente el valor de la función y su derivada en x . Esto permite calcular derivadas sin implementar un derivador simbólico general.

La extensión hacia una calculadora no cambia ese eje, sino que lo vuelve más usable. El parser permite escribir expresiones en una sintaxis cercana a la matemática usual; el modo archivo permite correr varios casos; el manejo de errores evita resultados silenciosos como `NaN`; y la mónada `EvalM` ordena el uso de entornos y fallos dentro del evaluador.

Desde el punto de vista de Análisis de Lenguajes de Programación, el interés del trabajo no está únicamente en calcular un resultado numérico. El proyecto permite discutir las etapas habituales de un lenguaje pequeño: una sintaxis concreta escrita por el usuario, una representación abstracta interna, una o más semánticas definidas sobre esa representación y una capa de interacción que usa el lenguaje sin confundirse con él. Esta separación es importante porque evita que el programa sea sólo una colección de funciones matemáticas sueltas.

2 Alcance y objetivos

El alcance fue definido para que el proyecto sea suficientemente rico desde el punto de vista de la materia, pero sin desviarse hacia un CAS completo. Se buscó representar expresiones, parsearlas desde texto, evaluarlas en un punto y obtener derivadas por diferenciación automática. También se incorporaron funciones matemáticas usuales, constantes, optimizaciones simples y lectura de archivos.

Un objetivo importante fue que las mejoras no fueran solamente acumulación de funcionalidades. Por eso se trabajó sobre robustez: el parser consume toda la entrada, el evaluador valida

dominios, las optimizaciones no ocultan errores y los tests cubren casos borde. Además, se separó el núcleo del lenguaje de la interfaz de consola para que el DSL pueda razonarse y probarse sin depender del ejecutable.

El soporte multivariable existe como base en la API mediante entornos, pero el modo interactivo y el formato de archivo siguen centrados en la variable `x`. Esta decisión mantiene simple la experiencia de usuario y deja una extensión posible sin hacer crecer demasiado el alcance actual.

En consecuencia, los objetivos pueden leerse en dos niveles. En el nivel matemático, se busca evaluar expresiones y obtener derivadas por números duales. En el nivel de lenguaje, se busca diseñar una sintaxis, un AST, un parser, interpretaciones y mecanismos de validación. Esta segunda lectura es la que justifica que el trabajo sea presentado como DSL/EDSL y no sólo como una calculadora.

3 Organización del proyecto

El código está dividido en módulos con responsabilidades separadas. Esta separación es parte del diseño del lenguaje: el AST no sabe cómo se evalúa, el parser no calcula resultados, el pretty printer no modifica semántica y la consola sólo actúa como interfaz.

El módulo `Expr` define la sintaxis abstracta y las optimizaciones conservadoras. `Parser` transforma texto en expresiones. `Evaluator` interpreta esas expresiones tanto sobre `Double` como sobre números duales. `EvalM` define la mónada usada para combinar entorno y errores. `PrettyPrinter` convierte expresiones nuevamente a texto intentando evitar paréntesis innecesarios. `FileReader` procesa archivos con varias expresiones. `NumericPolicy` concentra tolerancias numéricas compartidas. Finalmente, `DSL` funciona como punto de entrada público del lenguaje y reexporta las piezas principales.

La interfaz queda en `app/`. `Main` decide si se ejecuta el modo interactivo, el modo archivo o la ayuda. `Console` contiene la interacción con el usuario. Esta división deja claro que el núcleo del proyecto está en el lenguaje y sus interpretaciones, no solamente en el programa de consola.

Esta organización también reduce acoplamiento. Si se modifica la forma de mostrar resultados por consola, el AST y el evaluador no deberían cambiar. Del mismo modo, si se agrega una nueva función matemática al lenguaje, los cambios esperables están localizados en el AST, parser, evaluador, pretty printer y tests. Esa trazabilidad entre módulos ayuda a mantener el proyecto y hace más simple explicar el impacto de cada extensión.

4 Diseño del DSL/EDSL

El proyecto puede entenderse como una combinación de DSL y EDSL. Es un DSL porque define una sintaxis concreta propia para el usuario, por ejemplo `sin(x) + x^2`. Esa sintaxis se parsea desde texto y tiene reglas particulares de precedencia, funciones reservadas y comentarios. A la vez, tiene rasgos de EDSL porque la representación interna está embebida en Haskell mediante el tipo algebraico `Expr`; las expresiones también pueden construirse directamente desde código Haskell usando constructores.

Esta doble lectura es útil para el trabajo. La sintaxis textual hace que el lenguaje sea cómodo para usar como calculadora, mientras que la representación embebida permite definir semánticas de manera declarativa. Por ejemplo, evaluar, derivar, imprimir y optimizar son recorridos distintos sobre el mismo AST. Esa reutilización de la estructura interna es una de las razones principales para no evaluar directamente sobre strings.

Además, el diseño mantiene una frontera clara entre sintaxis y semántica. El parser sólo decide si una cadena pertenece al lenguaje y qué árbol representa. El evaluador decide qué significa ese árbol en un entorno dado. El pretty printer decide cómo volver a escribirlo como texto. Separar esas responsabilidades permite probar cada parte y detectar mejor dónde aparece un error.

5 Representación de expresiones

El núcleo del DSL es el tipo algebraico **Expr**. Este tipo representa la sintaxis del lenguaje: literales, variables, operadores binarios y funciones unarias. La estructura es explícita, lo cual permite escribir interpretaciones por pattern matching.

```
data Expr
  = Lit Double
  | Var String
  | Add Expr Expr
  | Sub Expr Expr
  | Mul Expr Expr
  | Div Expr Expr
  | Pow Expr Expr
  | Sin Expr
  | Cos Expr
  | Tan Expr
  | Sinh Expr
  | Cosh Expr
  | Tanh Expr
  | Arsinh Expr
  | Arcosh Expr
  | Artanh Expr
  | Log Expr
  | Exp Expr
  | Sqrt Expr
```

Esta representación tiene una ventaja central: separa la sintaxis de sus usos. La expresión `Add (Var "x") (Lit 1)` no decide si va a imprimirse, evaluarse u optimizarse; sólo describe una forma del lenguaje. Luego cada módulo puede recorrerla con su propia interpretación.

La elección de un AST algebraico también facilita ubicar invariantes. Por ejemplo, el optimizador trabaja sobre **Expr** y debe devolver otra **Expr** equivalente en los casos donde se aplica una regla. Si una regla puede cambiar el comportamiento frente a errores, no se aplica.

Otra ventaja de esta representación es que el compilador de Haskell ayuda a revisar la cobertura de casos. Cuando se agrega un constructor nuevo, como una función matemática adicional, aparecen puntos concretos del programa que deben actualizarse. En particular, el parser debe reconocer la función, el evaluador debe definir su semántica, el pretty printer debe saber imprimirla y los tests deben cubrirla. Esto hace que la evolución del lenguaje sea más controlada.

La desventaja de un AST explícito es que cada nueva forma del lenguaje implica tocar varios módulos. En este proyecto esa repetición se acepta porque el conjunto de construcciones es acotado y porque favorece la claridad. Para un lenguaje mucho más grande podría convenir otro diseño, pero para el alcance actual el tipo algebraico permite una explicación directa y formal.

6 Parser

El parser está implementado con Parsec. Su tarea es convertir una entrada textual, como `sin(x) + x^2`, en un valor de tipo **Expr**. Para eso se definió un lexer con operadores, paréntesis, identificadores, palabras reservadas, números y comentarios.

Una decisión importante fue definir explícitamente la precedencia:

potencia > menos unario > multiplicación/división > suma/resta

Esto evita ambigüedades en expresiones frecuentes. Por ejemplo, $-x^2$ se interpreta como $-(x^2)$, que coincide con la convención matemática usual. En cambio, $(-x)^2$ fuerza otra estructura mediante paréntesis.

También se exige que el parser consuma toda la entrada. Esto es relevante porque un parser que acepta prefijos podría considerar válida una cadena como:

```
x + 1 basura
```

En ese caso la parte `x + 1` podría parsearse bien, pero quedaría texto sobrante. El proyecto rechaza esa entrada para evitar resultados inesperados. Esta decisión vuelve el DSL más estricto y facilita detectar errores del usuario.

También se decidió reservar los nombres de funciones y constantes. Esto evita que `sin`, `log`, `pi` o `e` sean tratados accidentalmente como variables comunes. La elección simplifica el lenguaje de usuario y permite que expresiones como `sin(pi/2)` tengan una interpretación estable.

El soporte de comentarios en el lexer y en el lector de archivos cumple una función práctica. No cambia la semántica matemática, pero permite construir archivos de ejemplo más legibles. Desde el punto de vista de reproducibilidad, esto importa porque los casos de prueba pueden documentarse sin salir del formato ejecutable por el programa.

7 Números duales

Un número dual tiene la forma:

$$a + b\varepsilon$$

con:

$$\varepsilon^2 = 0$$

Si se evalúa una función diferenciable en $x + \varepsilon$, se obtiene:

$$f(x + \varepsilon) = f(x) + f'(x)\varepsilon$$

En el código se representa con:

```
data Dual = Dual
  { primal :: Double
  , deriv  :: Double
  }
```

La parte `primal` almacena el valor numérico y la parte `deriv` almacena la derivada respecto de la variable elegida. Para derivar respecto de `x`, el evaluador usa `Dual x 1`. Si una variable se considera constante, se le asigna derivada cero.

Las instancias `Num`, `Fractional` y `Floating` de `Dual` permiten expresar reglas de derivación de forma natural. Por ejemplo, la multiplicación implementa la regla del producto, y las funciones como seno, coseno o exponencial propagan la derivada según sus reglas conocidas. Aun así, el evaluador no delega ciegamente todo en esas instancias: para operaciones con restricciones de dominio se usan funciones seguras.

Esta técnica tiene una diferencia importante con la derivación simbólica. La derivación simbólica construiría una nueva expresión que representa f' . En cambio, los números duales calculan el valor de $f(x)$ y $f'(x)$ durante la evaluación. Esto evita implementar reglas de reescritura simbólica generales y reduce el alcance del proyecto, pero sigue mostrando un método real de diferenciación automática.

La implementación con type classes también es relevante. Al hacer que `Dual` sea instancia de clases numéricas, muchas operaciones pueden escribirse de manera parecida a como se escriben para `Double`. Esto muestra una ventaja idiomática de Haskell: se puede cambiar el dominio de interpretación de una expresión sin reescribir por completo las reglas algebraicas.

8 Evaluación escalar y evaluación dual

El módulo `Evaluator` contiene las dos interpretaciones principales del lenguaje. La evaluación escalar calcula únicamente el valor de la expresión:

```
eval :: Expr -> Double -> Either ErrorType Double
```

La evaluación dual calcula valor y derivada en una sola pasada:

```
evalDual :: Expr -> Double -> Either ErrorType Dual
```

Ambas funciones devuelven `Either ErrorType ...` porque la evaluación puede fallar. Esta decisión evita mezclar errores del dominio matemático con excepciones parciales de Haskell. Por ejemplo, dividir por cero, evaluar `log(0)` o intentar `sqrt(-1)` no debería producir un resultado silencioso ni cortar el programa sin control.

Además existen versiones con entorno:

```
evalWithEnv :: EvalEnv Double -> Expr -> Either ErrorType Double
```

```
evalDualWithEnv :: EvalEnv Dual -> Expr -> Either ErrorType Dual
```

Estas funciones desacoplan al evaluador de una única variable fija. El modo usuario sigue usando `x`, pero internamente el diseño ya permite expresiones con otras variables si se provee el entorno correspondiente. Para derivar respecto de `x` en una expresión con `x` e `y`, se puede usar `Dual x 1` para `x` y `Dual y 0` para `y`.

La evaluación escalar y la dual comparten la misma idea estructural: recorrer el AST y combinar resultados. La diferencia está en el dominio de los valores. En la evaluación escalar, los literales y variables producen `Double`; en la dual producen `Dual`. Esta separación permite comparar ambos resultados y detectar errores específicos de derivación, como derivadas no finitas en puntos donde el valor de la función sí existe.

Un ejemplo claro es `sqrt(x)` en $x = 0$. Como función real, el valor está definido y es cero. Sin embargo, su derivada no es finita en ese punto. Por eso el evaluador dual puede rechazar un caso que la evaluación escalar acepta. Esta distinción es importante para no confundir existencia del valor con existencia de la derivada.

9 Mónada de evaluación

Para ordenar el manejo de variables y errores se definió una mónada propia:

```
newtype EvalM env a =  
  EvalM (EvalEnv env -> Either ErrorType a)
```

Una computación `EvalM env a` recibe un entorno de variables y puede devolver un error o un valor. Conceptualmente combina dos ideas vistas en la materia: lectura de contexto, similar a `Reader`, y propagación de errores, similar a `Either`. Se implementó a mano para que las instancias sean visibles y para que el trabajo muestre claramente qué efecto se está modelando.

La función que ejecuta una computación es:

```
runEvalM :: EvalEnv env -> EvalM env a -> Either ErrorType a
```

El operador `>=` permite encadenar evaluaciones donde una etapa depende del resultado anterior. Por ejemplo, en una división primero se evalúa el numerador, luego el denominador y recién después se decide si se puede dividir. Si alguna parte falla, el error se propaga sin seguir calculando.

El proyecto también define `return = pure` en la instancia de `Monad`. Esto respeta la jerarquía actual de clases de Haskell, donde toda mónada también debe ser aplicativa. En el evaluador se usan `return` y `>=` para que el uso monádico sea explícito, especialmente en las operaciones donde hay validaciones intermedias.

La instancia `Alternative` agrega una forma simple de expresar respaldos:

```
lookupVar "z" <|> lookupVar "x"
```

Si la primera búsqueda falla, se intenta la segunda con el mismo entorno. Esto no modifica el lenguaje visible para el usuario, pero muestra cómo la mónada puede componer efectos de manera controlada.

La elección de implementar una mónada propia, en lugar de usar directamente una combinación ya existente, tiene una motivación didáctica. El tipo `EvalM` deja visible qué información circula por la evaluación: un entorno y un posible error. Esto permite relacionar el código con los contenidos de la materia sin esconder la estructura detrás de una abstracción más general.

Desde el punto de vista formal, `return` introduce un valor que no consulta el entorno ni falla, mientras que `>=` secuencia dos computaciones en el mismo entorno y corta la ejecución si aparece un error. Esa semántica coincide con lo que necesita el evaluador: cada subexpresión se evalúa bajo las mismas variables disponibles, pero cualquier fallo debe propagarse inmediatamente.

10 Manejo de errores

Los errores se representan con un tipo propio:

```
data ErrorType
  = DivideByZero
  | UndefinedVariable String
  | DomainError String
```

Esta decisión permite distinguir tipos de fallo. No es lo mismo que falte una variable a que una operación esté fuera del dominio real. Esa diferencia se usa tanto para testear como para mostrar mensajes más claros al usuario.

La función `mostrarError` separa la representación interna del mensaje final. El `Show` del tipo queda disponible para depuración, mientras que la interfaz puede presentar textos pensados para una persona que está usando la calculadora.

En el caso de números duales hay operaciones especialmente delicadas. `sqrt(0)` existe como valor real, pero su derivada no es finita si la entrada varía. `arcosh(1)` tiene un problema similar. Por eso el evaluador usa funciones como `safeSqrtDual`, `safeAcoshDual`, `safeAtanhDual`, `safeLogDual` y `safePowDual`. Estas funciones validan el dominio antes de llamar a las operaciones de `Floating`.

Separar los errores en un tipo algebraico también mejora los tests. Una prueba no necesita comparar solamente strings; puede verificar que el resultado sea `DivideByZero` o `DomainError`. Esto vuelve la validación menos frágil y permite cambiar el mensaje mostrado al usuario sin romper la semántica esperada del evaluador.

El uso de `mostrarError` completa esa separación. La interfaz no depende directamente de `show` sobre el error interno, sino de una función pensada para presentar mensajes. Así se distingue entre representación de datos y forma de comunicación con el usuario.

11 Pretty printer

El módulo `PrettyPrinter` toma una `Expr` y genera una representación textual legible. No se limita a poner paréntesis alrededor de todo, porque eso produce salidas correctas pero poco naturales. Tampoco puede eliminar paréntesis sin cuidado, porque podría cambiar la estructura de la expresión.

La solución usada es imprimir según niveles de precedencia. Una suma se imprime con menor precedencia que una multiplicación, y una potencia requiere atención especial en su base y exponente. Por ejemplo, una base negativa en una potencia debe conservar paréntesis:

```
(-1) ^ 2
```

También se controlan casos como potencias anidadas, restas a derecha y divisiones a derecha. La idea central es que el pretty printer debe ser agradable, pero ante todo debe preservar significado.

Para validar esta decisión se agregaron tests de roundtrip. Se imprime una expresión, se vuelve a parsear y se compara su comportamiento. Esta prueba es importante porque conecta dos módulos distintos: si el pretty printer genera una cadena ambigua, el parser puede reconstruir una expresión diferente.

El pretty printer no intenta ser una herramienta estética independiente, sino una parte más del lenguaje. Su salida puede volver a ser entrada del parser. Por eso el criterio de corrección no es sólo que el texto sea legible, sino que preserve la estructura semántica de la expresión. Este punto justifica especialmente los tests con potencias, negativos y paréntesis.

12 Optimización conservadora

El módulo `Expr` también contiene `optimize`. Esta función aplica reglas algebraicas simples sobre el AST, como eliminar sumas por cero, multiplicaciones por uno o plegar operaciones entre literales.

```
x + 0 = x
x * 1 = x
x / 1 = x
x^1 = x
2 + 3 = 5
```

La palabra clave es “conservadora”. Algunas identidades parecen correctas en álgebra, pero no son seguras dentro de este evaluador porque pueden ocultar errores. Por ejemplo, $x/x = 1$ falla si $x = 0$, y $x^0 = 1$ no debe ocultar el caso 0^0 . Por eso esas reglas no se aplican de manera general.

Esta decisión hace que el optimizador sea menos agresivo, pero más correcto respecto de la semántica observable del lenguaje. El objetivo no es simplificar todo lo posible, sino no transformar una expresión que debería fallar en otra que aparenta ser válida.

La invariante principal del optimizador puede formularse así: si una expresión original produce un error en un entorno determinado, el optimizador no debería eliminar artificialmente ese error mediante una regla algebraica que no sea válida en todos los casos del dominio. Esta invariante es más importante que obtener expresiones mínimas. En un lenguaje con errores observables, preservar errores también forma parte de preservar significado.

Por ese motivo, el plegado de constantes también se realiza con cuidado. Una división literal sólo se pliega si el denominador no es cercano a cero, y una potencia literal sólo se calcula si queda dentro del dominio real y produce un resultado finito. Estas condiciones conectan el optimizador con la política numérica del proyecto.

13 Procesamiento de archivos

El módulo `FileReader` permite ejecutar varias expresiones desde un archivo. Cada línea evaluable usa el formato:

```
expresion @ valor
```

El lado izquierdo se parsea como expresión del DSL y el lado derecho se interpreta como valor para `x`. Ese valor puede ser un número directo o una expresión constante, por ejemplo `pi/2`. Esto permite escribir ejemplos más naturales para funciones trigonométricas.

El lector también acepta comentarios de línea con `-` y comentarios de bloque con `{- ... -}`. Esta característica no es central matemáticamente, pero mejora la usabilidad de los archivos de ejemplo y permite registrar casos de prueba con contexto.

El modo archivo además funciona como puente entre el lenguaje y la validación. Los ejemplos no son solamente documentación informal: la suite de tests los recorre para verificar que sigan parseando y evaluando de forma controlada. Esto reduce el riesgo de que un cambio en el parser o en el evaluador deje obsoletos los casos de uso ya incorporados al proyecto.

14 Política numérica

El módulo `NumericPolicy` concentra tolerancias y criterios numéricos. En un evaluador con `Double`, comparar contra cero de forma exacta no siempre es una buena idea. Por eso se usan funciones como `esCasiCero`, `esEntero` y `esFinito`.

Centralizar estos criterios evita que aparezcan constantes mágicas dispersas por el código. Además hace más transparente el comportamiento del evaluador: cuando una operación se considera cercana a cero o cuando un exponente se considera entero, esa decisión está nombrada y localizada.

Esta centralización también evita inconsistencias entre módulos. Si el evaluador considera que un valor es cercano a cero con un criterio, pero los tests u optimizaciones usan otro, podrían aparecer comportamientos difíciles de explicar. Al concentrar esas decisiones, el proyecto gana coherencia y facilita futuros ajustes.

15 Interfaz pública y ejecutable

El módulo `DSL` reexporta las piezas principales del lenguaje. Su función es ofrecer una API limpia para ejemplos, tests o usos futuros. Desde ahí se puede acceder al AST, parser, evaluador, mónada de evaluación, pretty printer y procesamiento de archivos.

El ejecutable queda separado en `app/`. `Main` interpreta argumentos y decide entre modo interactivo, modo archivo o ayuda. `Console` maneja la interacción: lee una expresión, pide el valor de `x`, evalúa y muestra resultados. Esta separación evita que la lógica del lenguaje quede mezclada con entrada/salida.

Esta distinción también mejora la mantenibilidad. Las funciones del núcleo pueden probarse con `HUnit` sin simular entrada por teclado. La consola, en cambio, puede cambiar su formato de impresión o sus mensajes sin modificar la semántica del DSL. Para un proyecto de lenguaje, esta frontera es una decisión de diseño importante.

16 Invariantes de diseño

A lo largo del proyecto se mantuvieron varias invariantes. La primera es que el parser debe rechazar entradas ambiguas o parcialmente consumidas. La segunda es que el pretty printer debe producir texto que pueda volver a parsearse sin cambiar el significado observable de la expresión.

La tercera es que el optimizador no debe ocultar errores que el evaluador debería reportar. La cuarta es que los errores matemáticos deben representarse como valores de `ErrorType` y no como excepciones no controladas.

Estas invariantes no aparecen aisladas. Se refuerzan entre sí: el parser estricto evita aceptar basura, el pretty printer se valida con roundtrip, el optimizador se prueba con casos donde una regla agresiva sería incorrecta y el evaluador dual valida dominios antes de llamar a operaciones parciales. Por eso la robustez del proyecto no depende de una sola función, sino de varias decisiones coordinadas.

17 Comparación con un CAS simbólico

Un sistema de álgebra computacional apunta a manipulación simbólica general: derivación simbólica, integración, simplificación avanzada y transformación algebraica. Este proyecto toma otro camino. Trabaja con expresiones, pero su derivación se basa en evaluación con números duales, no en construir una nueva expresión derivada.

Esta diferencia delimita el alcance técnico del proyecto. El trabajo no intenta resolver todos los problemas de álgebra simbólica. Se concentra en mostrar cómo un lenguaje pequeño puede tener varias interpretaciones bien separadas y cómo los números duales permiten obtener derivadas de manera automática y composicional.

En particular, no se busca producir una fórmula simplificada para la derivada. Si se evalúa $\sin(x^2)$ en un punto, el sistema devuelve el valor y la derivada en ese punto, no una expresión simbólica equivalente a $2x \cos(x^2)$. Esta limitación es deliberada y coherente con la técnica elegida.

18 Tests y validación

La suite de tests tiene 138 casos. No sólo prueba ejemplos felices; también cubre errores de dominio, división por cero, variables no definidas, precedencia del parser, negativos, potencias, funciones hiperbólicas, pretty printer, roundtrip parser/pretty printer, optimizaciones conservadoras, `EvalM` y archivos de ejemplo.

Una parte relevante es la verificación automática de la carpeta `examples/`. La suite recorre los archivos `.txt`, parsea sus líneas evaluables y comprueba que cada evaluación termine de forma controlada, ya sea con un resultado o con un error esperado. Esto ayuda a que los ejemplos distribuidos junto con el proyecto no queden desactualizados respecto del código.

La validación actual se realiza con:

```
cabal check
cabal build all
cabal test
cabal exec -- runghc -isrc test\TestSuite.hs
cabal sdist
```

Además, los documentos principales se mantienen en LaTeX para poder compilar la guía y el informe final como PDF.

La validación combina pruebas unitarias con casos de integración. Las pruebas unitarias revisan comportamientos puntuales, como precedencia o errores de dominio. Los tests de archivo ejercitan el flujo completo: lectura de línea, limpieza de comentarios, parsing, optimización, evaluación escalar y evaluación dual. Esta combinación es importante porque algunos errores sólo aparecen cuando los módulos interactúan.

19 Limitaciones

El proyecto mantiene algunas limitaciones intencionales. No hay derivación simbólica general ni integración. Las optimizaciones son simples y conservadoras. El modo interactivo y el modo archivo trabajan con una variable principal x . El soporte multivariable está disponible en la API, pero no aparece todavía como lenguaje completo de usuario.

Estas limitaciones no son solamente faltantes, sino decisiones de alcance. Permiten que el trabajo se mantenga enfocado en el DSL, la evaluación segura, los números duales y el uso de mónadas.

También hay una limitación propia del uso de `Double`: los resultados numéricos están sujetos a aproximación de punto flotante. El proyecto no intenta implementar aritmética exacta. En cambio, define tolerancias para tomar decisiones razonables en validaciones de dominio y comparaciones numéricas. Esta elección es adecuada para una calculadora de expresiones reales, pero debe mencionarse como parte del alcance.

20 Trabajo futuro

Las extensiones más naturales serían permitir elegir la variable de derivación desde la entrada de usuario, ampliar el formato de archivo para entornos multivariados y mejorar la recuperación de errores del parser con mensajes más orientados a posición y contexto. También podrían agregarse propiedades de testeo más generales para parser, pretty printer y optimizador.

Otra línea posible sería sumar más funciones matemáticas, siempre que no desvíen el objetivo principal. En este proyecto conviene que cada extensión mantenga la regla de diseño usada hasta ahora: agregar funcionalidad sólo si mejora la robustez, la claridad o la expresividad del lenguaje.

21 Conclusión

El proyecto presenta un lenguaje pequeño de expresiones matemáticas implementado en Haskell. El AST permite separar sintaxis de interpretación; el parser acerca el lenguaje al uso textual; el evaluador escalar calcula valores; el evaluador dual calcula derivadas automáticamente; y `EvalM` ordena entorno y errores dentro de una mónada propia.

La parte más importante del diseño es que las decisiones se sostienen entre sí. El pretty printer se valida contra el parser, el optimizador respeta los errores del evaluador, las tolerancias numéricas están centralizadas y los ejemplos forman parte de la suite. El resultado es un trabajo acotado, pero robusto y coherente con los contenidos de la materia.